

PHOENIX Project — CY015 Attack Testing

Test Cases Document

1. Overview

This document contains the attack test cases prepared for the CY015 attack testing phase of the PHOENIX project. The test cases are built directly on the STRIDE threat analysis completed in CY005 with each test case takes a specific STRIDE threat that was already identified and theoretically documented and defines exactly how that threat will be tested against the real PHOENIX API.

The three endpoint groups assigned are the Threat Intelligence endpoints, the Risk Assessment endpoints, and the Dashboard endpoints. A total of ten test cases is defined across these groups. All STRIDE threat IDs referenced in this document correspond directly to the threats identified in the CY005 STRIDE analysis document.

2. Endpoints Being Tested

The following three endpoint groups are based on the CY015 Attack Testing Plan and the weekly team update from Himanshu Khanna.

Endpoint Group	Endpoints	Test Cases	STRIDE IDs Referenced
Threat Intelligence	GET /api/users/threats GET /api/users/threats/:threatId	RT-01, RT-02, RT-03	ST-03, ST-10, ST-17, ST-26
Risk Assessment	GET /api/users/risk-assessments GET /api/users/risk-assessments/:assessmentId	RT-04, RT-05, RT-06	ST-11, ST-17, ST-26
Dashboard	GET /api/users/dashboard/overview GET /api/users/dashboard/charts GET /api/users/dashboard/activity	RT-07, RT-08, RT-09, RT-10	ST-03, ST-08, ST-25, ST-26

3. Test Scenarios

The following test scenarios are derived from the STRIDE threat modelling (CY005) and represent potential real-world attack situations that the PHOENIX system must defend against. These scenarios guide the development of detailed test cases and ensure that all critical threats are validated during the attack testing phase.

Threat Intelligence Scenarios

- An attacker attempts SQL injection through query parameters to extract all threat intelligence records from the database
- An unauthorised End User attempts to access restricted threat details that should only be available to Analysts
- An unauthenticated attacker attempts to access threat data without providing a valid token

Risk Assessment Scenarios

- An attacker manipulates query parameters to access or modify risk assessment data beyond their authorised scope
- A low-privilege user attempts to access detailed risk assessment records that require higher privileges
- An attacker attempts IDOR (Insecure Direct Object Reference) by changing assessment IDs to access other users' data

Dashboard Scenarios

- An unauthenticated attacker attempts to access the dashboard without login credentials
- An attacker attempts to use an expired or stolen token to maintain access to the system
- An End User attempts to access Analyst-level dashboard data and visualisations
- An attacker manipulates dashboard filter parameters to retrieve unauthorised data or alter system output

These scenarios ensure that the system is tested against realistic attack behaviours identified in the STRIDE analysis and help validate the effectiveness of implemented security controls.

4. Test Cases

Each test case below follows the standard CY015 format. STRIDE threat IDs are referenced directly from the CY005 STRIDE analysis so all threat modelling and testing work links together.

3.1 Threat Intelligence Endpoint Tests

RT-01 — SQL Injection — Threat List Endpoint	
Test ID	RT-01
Endpoint Tested	GET /api/users/threats
Attack Tactic	SQL Injection via Query Parameters
Tool Used	SQL Map, Burp Suite
STRIDE Reference	ST-10 — Database Tampering (Critical Risk) — identified in CY005 STRIDE analysis as malicious code reaching the database layer via unvalidated query input
What We Did	1. Log in with Analyst credentials via POST /api/users/auth/login and copy the access_token 2. In Postman send GET /api/users/threats with Authorization: Bearer {{analyst_token}} 3. Open Burp Suite and intercept the same request 4. Inject SQL payload into the search/filter query parameter: ' OR '1'=1 5. Also test: ' OR 1=1-- and 1'; DROP TABLE threats; -- 6. Forward each modified request and record the HTTP status code and response body 7. Run SQLMap: sqlmap -u '{{base_url}}/api/users/threats?search=test' --headers='Authorization: Bearer {{analyst_token}}' --dbs 8. Note whether any database content, table names, or error messages are returned
Expected Result	400 Bad Request or 422 Unprocessable which SQL payload must be rejected with no threat data or database information returned in the response body.
Actual Result	(To be completed after testing)
Screenshot	(Attach screenshot)
Status	Pending
Recommendation	If this test fails, the API is not sanitising query inputs before passing them to the database. Implement parameterised queries (prepared statements) on all threat list endpoint database interactions as specified in Mitigation M-19 of the CY005 Map Mitigations document.

RT-02 — RBAC Enforcement — Threat Detail Endpoint (End User Access)	
Test ID	RT-02
Endpoint Tested	GET /api/users/threats/:threatId
Attack Tactic	RBAC Access Control Testing — End User accessing Analyst-restricted data
Tool Used	Postman
STRIDE Reference	ST-17 — Information Disclosure (Critical Risk) + ST-26 — Elevation of Privilege (Critical Risk) — identified in CY005 STRIDE analysis as End User role accessing restricted threat intelligence records
What We Did	1. Log in as Analyst and send GET /api/users/threats to retrieve a list of valid threat records 2. Copy one valid threatId value from the response 3. Log out and log in with End User credentials — copy the end_user_token 4. In Postman send GET /api/users/threats/{threatId} with Authorization: Bearer {end_user_token} 5. Observe whether the full threat intelligence record is returned or whether access is denied 6. Repeat with 3 different threatId values to confirm the result is consistent 7. Record the HTTP status code and the content of the response body for each attempt
Expected Result	403 Forbidden for all attempts — the End User role must not be able to access individual threat intelligence detail records which are restricted to Analyst level and above per the RBAC permissions matrix
Actual Result	(To be completed after testing)
Screenshot	(Attach screenshot)
Status	Pending
Recommendation	If this test fails, the RBAC check is missing or not enforced on the detail endpoint. Enforce server-side role verification on every request to GET /threats/:threatId — rejecting any request from an account whose role is below Analyst level.

RT-03 — Unauthenticated Access — Threat List Endpoint	
Test ID	RT-03
Endpoint Tested	GET /api/users/threats
Attack Tactic	Unauthenticated Access Attempt — no token and invalid token
Tool Used	Postman
STRIDE Reference	ST-03 — Spoofing / Unauthenticated Access (Critical Risk) — identified in CY005 STRIDE analysis as an unauthenticated attacker accessing Phoenix threat intelligence without a valid identity.
What We Did	1. In Postman create GET {{base_url}}/api/users/threats 2. Send the request with NO Authorization header at all 3. Record the HTTP status code and response body 4. Send the same request again with a deliberately invalid token: Bearer faketoken12345 5. Record the HTTP status code and response body for this second attempt 6. Check whether any threat data or system information appears in any error messages
Expected Result	401 Unauthorized for both attempts — no threat data must be accessible without a valid authenticated session token
Actual Result	(To be completed after testing)
Screenshot	(Attach screenshot)
Status	Pending
Recommendation	If this test fails, the endpoint has no authentication gate. Add mandatory Bearer token verification as middleware on all /threats endpoints — any request without a valid unexpired token must be rejected with 401 before any data is returned.

3.2 Risk Assessment Endpoint Tests

RT-04 — Parameter Tampering — Risk Assessment List Endpoint	
Test ID	RT-04
Endpoint Tested	GET /api/users/risk-assessments
Attack Tactic	Parameter Tampering via Query Filter Fields
Tool Used	Burp Suite
STRIDE Reference	ST-11 — Correlation Engine Tampering (Critical Risk) — identified in CY005 STRIDE analysis as an attacker manipulating risk assessment inputs to corrupt hazard-to-threat correlation outputs.
What We Did	1. Log in with End User credentials and copy the end_user_token 2. In Postman set Burp Suite as the proxy and send GET /api/users/risk-assessments with end_user_token 3. Intercept the request in Burp Suite before it reaches the server 4. Modify query parameters — test each of these: ?role=admin, ?filter=all, ?access=unrestricted, ?user_id=1 5. Forward each modified request one at a time and record the response 6. Compare the returned data against what the End User is permitted to see per the RBAC matrix 7. Note whether the server ignores the tampered parameters or responds to them
Expected Result	403 Forbidden or only the End User's own permitted data returned — the server must validate all filter parameters server-side and must not trust client-supplied role or access level values
Actual Result	(To be completed)
Screenshot	(Attach screenshot)
Status	Pending
Recommendation	If this test fails, the server is trusting client-supplied query parameters to control data access scope. Implement server-side parameter validation that ignores or rejects any role or access-level parameters submitted by the client which access scope must be derived from the authenticated user's server-side session record only.

RT-05 — RBAC Enforcement — Risk Assessment Detail Endpoint (End User Access)	
Test ID	RT-05
Endpoint Tested	GET /api/users/risk-assessments/:assessmentId
Attack Tactic	RBAC Access Control Testing — End User accessing restricted assessment detail
Tool Used	Postman
STRIDE Reference	ST-26 — Elevation of Privilege (Critical Risk) — identified in CY005 STRIDE analysis as a user with a limited role accessing data restricted to a higher-privilege role
What We Did	1. Log in as Analyst and send GET /api/users/risk-assessments to retrieve a list of valid assessment records 2. Copy one valid assessmentId from the response 3. Log in with End User credentials and copy the end_user_token 4. In Postman send GET /api/users/risk-assessments/{{assessmentId}} with Authorization: Bearer {{end_user_token}} 5. Record the HTTP status code and response body 6. Repeat with 3 different assessmentId values to verify the result is consistent
Expected Result	403 Forbidden for all attempts — End User role must not be able to access individual risk assessment detail records
Actual Result	(To be completed)
Screenshot	(Attach screenshot)
Status	Pending
Recommendation	If this test fails role-based access control is not enforced on the assessment detail endpoint. Add server-side role verification that checks the authenticated user's role before returning any assessment detail reject with 403 if the role is below the minimum required level.

RT-06 — IDOR — Insecure Direct Object Reference on Risk Assessment Detail	
Test ID	RT-06
Endpoint Tested	GET /api/users/risk-assessments/:assessmentId
Attack Tactic	IDOR Testing — incrementally modifying record ID to access other users' records
Tool Used	Burp Suite, Postman
STRIDE Reference	ST-17 — Information Disclosure (Critical Risk) — identified in CY005 STRIDE analysis as sensitive risk scoring data being exposed to an unauthorised party through predictable ID enumeration.
What We Did	1. Log in as Analyst and note which assessmentId values your own account can legitimately access 2. In Burp Suite intercept GET /api/users/risk-assessments/{your_legitimate_id} 3. Modify the ID incrementally — test IDs: 1, 2, 3, 10, 99, 100 4. Also test boundary values: 0, -1, and a string value such as abc 5. Forward each modified request and record which IDs return 200 and which return 403 6. Check whether any returned records belong to a different user or session than your own
Expected Result	403 Forbidden for any assessmentId not belonging to the authenticated analyst's own records — object-level authorisation must be enforced on every record retrieval
Actual Result	(To be completed after testing)
Screenshot	(Attach screenshot)
Status	Pending
Recommendation	If this test fails, the API is not checking object ownership before returning records. Implement object-level authorisation checks that verify the requested assessment Id belongs to the authenticated user before any data is returned and predictable sequential IDs should not expose other users' records.

3.3 Dashboard Endpoint Tests

RT-07 — Unauthenticated Access — Dashboard Overview Endpoint	
Test ID	RT-07
Endpoint Tested	GET /api/users/dashboard/overview
Attack Tactic	Unauthenticated Access Attempt — no token and invalid token
Tool Used	Postman
STRIDE Reference	ST-03 — Spoofing / Unauthenticated Access (Critical Risk) — identified in CY005 STRIDE analysis as an unauthenticated attacker gaining access to the PHOENIX DMT dashboard without a valid identity
What We Did	1. In Postman create GET {{base_url}}/api/users/dashboard/overview 2. Send the request with NO Authorization header 3. Record the HTTP status code and response body 4. Send the same request again with an invalid token: Bearer invalidtoken999 5. Record the HTTP status code and response body for this second attempt 6. Check whether any dashboard data appears in the response or in error messages
Expected Result	401 Unauthorized for both attempts — the dashboard must be completely inaccessible without a valid authenticated session
Actual Result	(To be completed after testing)
Screenshot	(Attach screenshot)
Status	Pending
Recommendation	If this test fails, the dashboard overview endpoint has no authentication gate. Add mandatory Bearer token verification middleware by any request without a valid token must receive 401 before any dashboard data is processed or returned.

RT-08 — Expired Token — Dashboard Overview Endpoint	
Test ID	RT-08
Endpoint Tested	GET /api/users/dashboard/overview
Attack Tactic	Expired Token Access — testing whether the server enforces token expiry
Tool Used	Postman
STRIDE Reference	ST-25 — Session Exploitation (Critical Risk) — identified in CY005 STRIDE analysis as an attacker using a stolen or expired token to maintain persistent unauthorised access to the DMT dashboard
What We Did	1. Log in with any valid account and copy the access_token 2. Decode the JWT token (use jwt.io) to find the expiry timestamp in the payload 3. Wait for the token to expire naturally OR use a short-expiry test token provided by the backend team. 4. Send GET /api/users/dashboard/overview with the expired token in the Authorization header 5. Record the HTTP status code and response body 6. Note whether the server validates expiry or accepts the token regardless
Expected Result	401 Unauthorized — expired tokens must be rejected server-side with no dashboard data returned
Actual Result	(To be completed after testing)
Screenshot	(Attach screenshot)
Status	Pending
Recommendation	If this test fails, the server is not validating token expiry on dashboard requests. Implement server-side JWT expiry validation on every authenticated request — the server must verify the exp claim and reject any expired token with 401 regardless of what the client sends.

RT-09 — RBAC Enforcement — Dashboard Charts Endpoint (End User Access)	
Test ID	RT-09
Endpoint Tested	GET /api/users/dashboard/charts
Attack Tactic	RBAC Access Control Testing — End User accessing Analyst-level chart data
Tool Used	Postman
STRIDE Reference	ST-26 — Elevation of Privilege / RBAC Misconfiguration (Critical Risk) — identified in CY005 STRIDE analysis as overly permissive role assignment granting End User access beyond their authorised scope
What We Did	1. Log in as End User and copy the end_user_token 2. Send GET /api/users/dashboard/charts with Authorization: Bearer {{end_user_token}} 3. Record the full response body and note what data is visible 4. Log in as Analyst and send the same request with the analyst_token 5. Compare both responses — check whether the End User receives the same full chart data as the Analyst 6. According to the RBAC matrix End User has Limited dashboard access only — any full Analyst-level data is a failure
Expected Result	403 Forbidden or significantly limited data returned to End User — full Analyst-level chart data must not be visible to an End User account
Actual Result	(To be completed after testing)
Screenshot	(Attach screenshot)
Status	Pending
Recommendation	If this test fails RBAC is not enforced on the chart's endpoint. Add server-side role checking before returning chart data, then filter the response based on the authenticated user's role so End Users receive only the limited data they are authorised to see.

RT-10 — Parameter Tampering — Dashboard Activity Endpoint	
Test ID	RT-10
Endpoint Tested	GET /api/users/dashboard/activity
Attack Tactic	Parameter Tampering via Activity Filter Fields — attempting to access another user's activity data
Tool Used	Burp Suite
STRIDE Reference	ST-08 — Stream Processing Tampering (Critical Risk) — identified in CY005 STRIDE analysis as parameter manipulation causing the dashboard to return data belonging to a different user or session
What We Did	1. Log in as Analyst and copy the analyst_token 2. Open Burp Suite and set it as proxy — send GET /api/users/dashboard/activity with analyst_token 3. Intercept the request in Burp Suite before it reaches the server 4. Modify the user_id parameter to a different user's ID value 5. Try modifying date range parameters to very wide ranges or future dates 6. Try modifying any activity_type or category filter parameters to unexpected values 7. Forward each modified request and record whether the server returns the authenticated user's own data only or another user's activity data
Expected Result	403 Forbidden or only the authenticated analyst's own activity returned — the server must validate all filter parameters server-side and never trust client-supplied user_id values
Actual Result	(To be completed after testing)
Screenshot	(Attach screenshot)
Status	Pending
Recommendation	If this test fails, the server is trusting client-supplied parameters to determine which user's activity to return. The user_id must always be derived from the authenticated session on the server side which any client-supplied user_id parameter must be ignored entirely.

5. Summary

The following table provides a consolidated summary of all ten test cases. Status will be updated to Pass, Partial Pass, or Fail after testing is complete.

Test ID	Endpoint	Attack Tactic	STRIDE Ref	Tool	Status
RT-01	GET /api/users/threats	SQL Injection	ST-10	SQLMap, Burp Suite	Pending — awaiting test environment
RT-02	GET /api/users/threats/:threatId	RBAC — End User	ST-17, ST-26	Postman	Pending — awaiting test environment
RT-03	GET /api/users/threats	Unauthenticated Access	ST-03	Postman	Pending — awaiting test environment
RT-04	GET /api/users/risk-assessments	Parameter Tampering	ST-11	Burp Suite	Pending — awaiting test environment
RT-05	GET /api/users/risk-assessments/:id	RBAC — End User	ST-26	Postman	Pending — awaiting test environment
RT-06	GET /api/users/risk-assessments/:id	IDOR	ST-17	Burp Suite, Postman	Pending — awaiting test environment
RT-07	GET /api/users/dashboard/overview	Unauthenticated Access	ST-03	Postman	Pending — awaiting test environment
RT-08	GET /api/users/dashboard/overview	Expired Token	ST-25	Postman, Burp Suite	Pending — awaiting test environment
RT-09	GET /api/users/dashboard/charts	RBAC — End User	ST-26	Postman	Pending — awaiting test environment
RT-10	GET /api/users/dashboard/activity	Parameter Tampering	ST-08	Burp Suite	Pending — awaiting test environment